Decisions in Milliseconds

Jev and the rise of "System One" models

Reading companion to the AI Lunch talk by Hendrik Neumann, 2 October 2026. Facts as of 27 September 2026.

How to use this article

This article accompanies the slides of the talk. It follows the talk in order, contains everything the speaker notes say and adds background where a slide can only hint at something. Deep dives stay in the research document (`research.md`), which is much longer and organised by topic.

Numbers in square brackets, such as [5], point to the numbered sources at the end. The numbering is identical to the research document, so you can look a source up there as well. A few sections end with a pointer to the matching section of `research.md`. All facts are as of 27 September 2026; Jev and its ecosystem change quickly.

Four ways to read it

Time	Path
about 5 min	"The short version", then sections 7 and 8
about 10 min	"The short version", then sections 1, 6, 7 and 8
about 20 min	The talk: sections 1 to 11
about 26 min	Everything, including the background chapter

Sections at a glance

Section	Slides	Reading time
The short version	-	1 min
1. What Jev is	Slides 2 to 7	3 min
2. Who is behind it, and why the name	Slide 8	1 min
3. "Can't hallucinate": what the claim means	Slide 9	1 min
4. Earlier work	Slide 10	2 min
5. The open ecosystem after two weeks	Slide 11	2 min
6. What independent tests found	Slides 12 to 15	4 min
7. The pattern: System 1 plus System 2	Slide 16	1 min
8. Playbook for our projects	Slide 17	1 min
9. My take	Slide 18	1 min
10. The live demo: Kev on a MacBook	Slide 19	2 min
11. Questions to take away	Slide 20	1 min
12. Background chapter	Backup slides 22 to 24	6 min
Total		**about 26 min**

The short version

- **Jev is a model that cannot write a sentence.** You send it a piece of state, such as a ticket or an e-mail, plus typed questions. Within 70 to 500 milliseconds you get decisions back, each with a probability [5].
- **"Cannot hallucinate" is a schema guarantee.** The model can only return options you declared. It can still pick the wrong one, and a wrong but valid answer is harder to notice than broken JSON [9][30].
- **Earlier work exists.** Small scoring models and cascades were published years before Jev, and within two weeks about 35 open

projects copied its interface [12][24][52].

- **Independent tests give a mixed picture.** Asked one broad question, Jev scored 62.6 percent against 81.3 for Claude Haiku 4.5 [18]. Split into five narrow questions and fitted on labelled data, it reached 95 percent [19]. A small trained encoder beats it when labels exist [21].
- **The architecture is reusable.** A fast decision layer handles the routine cases, code acts when confidence is high, and a large model or a person takes over otherwise. Which model sits in that layer is an implementation detail [9][32].

---

1. What Jev is

Slides 2 to 7, reading time about 3 min

A model that cannot write

On 15 September 2026 the San Francisco startup TypeSafe AI launched Jev, a model that cannot write a single sentence [5]. You send Jev some state, for example a support ticket or an e-mail, together with a set of typed questions. Within 70 to 500 milliseconds you get decisions back, each with a probability [5]. Input costs 4.2 cents per million tokens and output is free [5].

Access has changed twice since launch. About a week after launch TypeSafe dropped its waitlist, and the rush briefly overloaded the API [64][5]. On 27 September the TypeSafe homepage says the service is invite-only again: new users join a waitlist, existing users can still log in [69].

One endpoint, three kinds of question

Technically there is one endpoint. Your application posts a block of state and a map of questions [4]. There are three question types [5] [4]:

- A **Choice** picks one of up to 255 options.
- A **Score** places something on a scale.
- A **Noul**, TypeSafe's own name, returns a yes-or-no probability.

All questions run in parallel and in isolation against the same state [4]. Isolation means that one question does not see the answers to the others.

One developer described Jev as a smart if-statement. Ordinary code branches on values it can compute. It breaks down when the condition is a judgement, such as whether a customer is angry. Jev supplies that judgement [36]. The division of labour is strict: your application defines the allowed answers and decides whether to act, and Jev only delivers the judgement [10]. With an LLM your code has to parse and validate free text; with Jev the answer arrives already typed.

One ticket, three questions

The schematic below shows one concrete call. The syntax is simplified and the numbers are illustrative.

```
state       "I was charged twice for my September invoice."

questions   team      Choice   billing | account | technical
            urgency   Score    1 (low) ... 5 (critical)
            security  Noul     is this a security issue?

answers     team      = billing    0.91
            urgency   = 3 of 5     0.78
            security  p(yes)       0.12
```

Three typed answers come back in one call: the ticket goes to billing at 0.91, the urgency is 3 out of 5 at 0.78, and the probability of a security issue is 0.12. The next three sections look at each type with its own example.

Choice: which team?

The state is again the support ticket about the double charge. The question asks which team should handle it, with three allowed options. The model returns one of them plus a probability for each, here billing at 0.91. A Choice can have up to 255 options [4]. The answer can only be one of the options the application declared, and code decides what happens next.

Score: how urgent?

The second example is a ticket written in capitals: "URGENT!!! The logo in the footer is two pixels off." The question places it on an urgency scale from 1, low, to 5, critical.

A keyword rule marks this ticket as urgent because the word is there. SAP's own reference app for incident management does exactly that: its handler sets urgency to high whenever the title matches "urgent" [59]. A useful decision model follows the content of the ticket: a cosmetic problem gets a low level however loudly it is written. In the illustration level 1 gets 0.41, with level 2 close behind at 0.38. The live demo tests this with a real model.

Noul: is this destructive?

The third example is a real output. A coding agent is asked to fix a typo in the README and proposes `git push --force origin main`. The question is whether this command is destructive. The answer comes from decider:2b, an open, Jev-compatible model served locally with Ollaya, and reads: probability 0.90 for yes, in 178 milliseconds on an RTX 4090 [60]. The same request also asked whether the command is on task, and the model said no [60]. Your code then blocks the command or asks a human. This is the guardrail use case: a cheap, fast check in front of an action that is hard to undo.

Deeper: `research.md`, section 3.

---

2. Who is behind it, and why the name

Slide 8, reading time about 1 min

TypeSafe AI was founded in 2024. One of its founders, Diogo Almeida, spent about four years at OpenAI working on RLHF, InstructGPT and ChatGPT [7]. His stated motivation, told to TechCrunch, is disappointment with conversational models: he sees "lightning in a bottle" that is not yet useful, because today's models are built for conversations with people and are awkward to use inside software [2]. DCVC led a 40-million-dollar seed round [6].

The name carries two references:

- **System One** refers to Daniel Kahneman's fast, intuitive way of thinking [10]. The background chapter explains what that means and where the comparison stops working.
- **Jev** refers to William Stanley Jevons, whose paradox says that when something gets cheaper, we use more of it [5]. The bet behind the name is that decisions will be made far more often once they are cheap enough.

Training is called RLCD, reinforcement learning for calibrated decisions, and uses purely synthetic data [2][7]. TypeSafe has not published how RLCD works, so the comparison with RLHF in the background chapter is an interpretation.

Officially, the architecture is undisclosed [33]. Outside observers suspect an open-weight language model as the base [33][4]. The most detailed outside attempt is an analysis by Archer Hume, who probed the API with about 10,000 calls. He infers that Jev keeps a pretrained LLM, encodes the shared state once, evaluates all questions in parallel without letting them see each other, and reads answer probabilities from internal representations instead of generating text [49]. This is inferred from black-box measurements and TypeSafe has not confirmed it. For consultants the practical consequence is that cost stability, model provenance and supply-chain risk cannot be assessed today.

Deeper: `research.md`, sections 4 and 5.

---

3. "Can't hallucinate": what the claim means

Slide 9, reading time about 1 min

TypeSafe says that Jev cannot hallucinate. That is a schema guarantee. Jev cannot return an option you did not declare, but it can pick the wrong valid option, and TypeSafe's own FAQ acknowledges this [9].

The zero-percent figure was not measured; it follows from the construction [4]. The structural part does hold up in practice. One large independent study counted zero invalid answers across more than 23,000 calls [20].

A wrong but valid answer is harder to detect than broken JSON, because it passes every check [30].

TypeSafe publishes a "jaggedness" page for each model version that lists known weaknesses [3]. For jev-1.13 these include reading instructions literally, trouble with extra levels of indirection and weak numeric precision [3]. Their advice is to keep arithmetic in code and not to use score outputs to compute exact magnitudes [3]. One write-up counts nine documented failure modes, among them

susceptibility to adversarial content in the state [11]. Tom's Hardware summarises that responsibility for acting on confidence values lies with the developer [8].

Deeper: `research.md`, section 8.

4. Earlier work

Slide 10, reading time about 2 min

Small scoring models, probability outputs and cascades of cheap and expensive models were published between 2019 and 2025.

When	Who	What
2019	Hugging Face	DistilBERT, a small encoder model [41]
2023	Stanford	FrugalGPT, cascades of cheap and expensive models, up to 98 % lower cost [24]
March 2025	Nandakishor Mukkunnoth	SalesRLAgent [12]
15 Sep 2026	TypeSafe AI	Jev
three days later	Nandakishor Mukkunnoth	Laya, 421M parameters, Apache 2.0 [14]

SalesRLAgent is a reinforcement-learning model trained on synthetic data that outputs a probability instead of text. It predicts the conversion probability of sales conversations [12]. It sat on Hugging Face under the name sales-conversion-model-reinf-learning [13]. It covers one vertical, sales, and shares the approach with Jev: non-generative, trained with reinforcement learning, probability as output, synthetic data.

Laya, released three days after Jev, is an open model with 421 million parameters that runs locally [14]. It is a small encoder with a decision head that scores options instead of generating them [14][15]. Its model card reports the limits: zero-shot it scores 0.362 where random guessing gets 0.318, and its calibration only becomes good after the temperature is refitted per question [14].

Older building blocks. In 2019 Hugging Face released DistilBERT [41]. Fine-tuned encoders of this kind still beat Jev today when labelled data exists [21]. The Stanford team behind FrugalGPT showed in 2023 that cascading cheap and expensive models can cut costs by up to 98 percent [24], and they used a small DistilBERT scorer to decide when to stop [25]. A Japanese analysis of seven Jev clones concludes that the clones reproduce only the computational form, and that this form has existed since about 2020 [34].

TypeSafe added the packaging: a general-purpose, hosted API with typed answers, probabilities and a price of 4.2 cents per million input tokens [5].

Deeper: `research.md`, sections 9 and 10.

5. The open ecosystem after two weeks

Slide 11, reading time about 2 min

Within two weeks, many groups had copied the interface. One catalogue counts about 35 open alternatives, all with self-reported numbers that nobody has re-run independently [52].

Most of them work the same way. They run one forward pass over the state, read out the scores for the allowed options and generate no tokens [51]. They copy the interface while TypeSafe's model and training stay closed [51]. Nobody has reproduced RLCD [53].

Project	What it is
Kev	Family of decision models on Qwen, 0.6 to 27 billion parameters, speaks TypeSafe's API [50][67][68]
Laya	421 million parameters, plus a 322-million multilingual checkpoint for over 100 languages [14][57]
Julia 1	144 million parameters, runs on a CPU, about 104 dollars of training [63]
Ollaya	Local runtime for open decision models behind Jev's API [60]
von, SemIf, NanoJev	Further local projects [52]
Tev1	Hosted by Together AI, answers only Choice questions [53]

Kev by Jared Palmer builds on Hume's analysis of Jev's architecture [49][50]. The official TypeSafe SDK works against a local Kev server, and the 4B and 9B models fit on a 32-gigabyte Mac [50]. On its own benchmark Kev-9B is 3.5 points behind Jev (0.822 against 0.857) and less well calibrated. The author stresses that this is not a controlled comparison, because Jev's training data is unknown [50]. Kev now ranges from 0.6 billion parameters, an older variant that is fast on Apple Silicon, up to 27 billion [67][68].

Julia 1 from a small Brazilian lab has 144 million parameters, runs on a CPU and cost about 104 dollars of GPU time to train [63]. Its authors report beating Jev's reference values on three of four tasks. On the fourth, a banking task with 72 categories, it scores 64 percent against 87 percent, and the Jev values were taken from a protocol, not re-measured [63].

Ollaya plays the role that Ollama plays for chat models. It is open source, downloads open decision models and serves them locally behind TypeSafe's API, so the official SDK runs unchanged [60]. Its own chart shows Laya at 8 to 10 milliseconds on an RTX 4090 against 236 to 276 milliseconds for the hosted Jev API, with the caveat that the setups differ and the comparison only gives an order of magnitude [60].

The frameworks followed quickly. Pydantic AI has a model class for Jev, and LangChain offers a router middleware that lets Jev choose between a fast and a strong model [65][66]. The multilingual Laya checkpoint matters for German client data [57].

A community list also contains projects that let Jev decide stock or crypto trades [55].

Deeper: `research.md`, sections 7 and 9.

6. What independent tests found

Slides 12 to 15, reading time about 4 min

Most of the numbers below come from independent authors who published on blogs or on GitHub, several of them from small studies. Treat them as indications.

TypeSafe advertises a decision path 193.6 times faster and 444.6 times cheaper than an LLM [11]. One analysis points out that the 444.6 figure uses the average of two other models as the reference answer, so it measures agreement with them and not accuracy [19]. Independent comparisons find about 12 to 27 times lower cost than Claude Haiku and roughly five times lower latency than GPT-5.6 Terra [19][22].

Phishing: one question against five

The phishing test is a reproducible benchmark with 2,000 e-mails [17].

2,000 e-mails	One question	Five questions plus labels
Jev	62.6 %	**95.0 %**
Claude Haiku 4.5	**81.3 %**	93.2 %
Two-line regex	n/a	about 92 %

Asked one question, "is this phishing?", Jev got 62.6 percent right and Claude Haiku 4.5 got 81.3 [18]. Jev caught only 43 percent of the actual phishing mails [30]. It was about three times faster and twelve times cheaper [18]. Part of the speed advantage is network time: 163 of Jev's 239 milliseconds were the network floor measured from France, so the ratio will not transfer to every setup [18][23].

The picture changes when you split the question into five narrow signals, for example whether a link points to free hosting, and fit weights on 1,000 labelled mails. On the other 1,000 mails Jev reaches 95 percent [19]. Haiku with the same five questions reaches 93.2 percent and a two-line regex about 92 percent; the differences are not statistically significant [20]. The accuracy comes from decomposing the question and adding labelled data [19].

Labels are the correct answers that a human has already attached to example data, such as 1,000 e-mails marked "phishing" or "fine".

Baselines: a small trained encoder wins

On the Banking77 intent benchmark one test measured Jev at 0.78 against 0.85 for GPT-5.6 Terra. Jev was five times faster and cost 4 cents against 2 dollars per thousand calls. It also counts about twice as many input tokens for the same text, which shrinks the price advantage [22].

A pre-registered study added a small trained baseline. A frozen encoder plus logistic regression, trained on 10,000 labelled examples, scored 0.933 on Banking77. That is better than the frontier LLM (0.875) and Jev (0.832), at 9 milliseconds and no per-call cost [21] [22]. The author recommends testing this baseline before paying per call. Without labels, Jev is a reasonable zero-shot router [21]. A

second, smaller test measured Jev at 0.87 [62]. Numbers depend on the setup, which is one more reason to measure on your own data.

Confidence: calibrate per question

The feature that is supposed to make Jev safe is its confidence value. If the model says 70 percent, it should be right in 70 percent of such cases. That property is called calibration. We have known since 2017 that neural networks are often poorly calibrated and that a simple post-processing step, temperature scaling, corrects much of the error [26].

TypeSafe publishes no calibration metrics for any dataset [20]. An independent study used synthetic support tickets with a priority rule that was not visible in the text. There, Jev was right 44.7 percent of the time while claiming 74 percent confidence on average [19]. The recommended practice is to calibrate per question on your own data, to run in shadow mode before going live and to scale thresholds with risk [19][11]. A read-only action may fire at a low threshold, a destructive one should wait for high confidence [11].

How much can you automate?

Coverage measures how much can be automated: what share of decisions can the model take while errors stay below 5 percent? Here the original still leads. In Kev's own comparison Jev covers 0.70 of decisions and Kev 0.45 to 0.57 [50]. The number also depends heavily on the task. In a small pilot with 100 examples per task, Jev covered 83 to 86 percent of news and banking-intent decisions and none of the emotion labels [62]. For emotions its accuracy was 0.48, and in 16 percent of the cases it gave the correct answer a probability of zero [62]. This is the number for a business case, and it has to be measured on the client's data.

Deeper: `research.md`, sections 11 and 12.

---

7. The pattern: System 1 plus System 2

Slide 16, reading time about 1 min

Most steps in an agent are decisions: route this ticket, is this input safe, which tool should run, does this need a human [32]. A System-1 layer makes the cheap decision. If the confidence is above a threshold calibrated for that question, code acts. Otherwise the case is escalated to a large model or to a person. The generative model plans and writes, and the decision model scores and routes [9].

Whether that layer is Jev, Kev, Laya, a trained encoder or a regex is an implementation detail that you choose per use case. Typical candidates in our projects are ticket and e-mail triage, document classification, pre-filtering for retrieval pipelines and safety checks before agent tool calls [32][9].

The cascade benefit is plausible, but the one pre-registered test could not yet prove it statistically: its confidence interval for the effect spanned zero [23].

A rule of thumb (our own synthesis, not validated). A decision fits a System-1 layer when it is frequent, repetitive, has stable categories and offers feedback to check against. A task that is novel, multi-step or needs an explanation stays with a large model or a human. The Kahneman and Klein test in the background chapter adds the questions to ask first.

Deeper: `research.md`, section 14.

---

8. Playbook for our projects

Slide 17, reading time about 1 min

1. **Baseline first.** Start with rules, a regex, or a supervised encoder if labelled data exists [21][20]. If a two-line regex reaches 92 percent, that is your bar.
2. **Decompose questions.** Turn vague questions into narrow, literal ones. "Does the link point to free hosting?" works better than "Is this phishing?" [17][3].
3. **Calibrate in shadow mode.** Calibrate thresholds per question and run in shadow mode on 1,000 to 2,000 decisions before going live [19].
4. **Pin the version.** Jev is at version 1.13, with known weaknesses published per version [61][65]. Answers can change between identical passes, in 1.33 and 2.2 percent of cases in two studies [20]. Run regression tests in CI; tools like `jevassert` record predictions and check accuracy, calibration and cost offline [35][31]. TypeSafe's own tutorial recommends pinning an exact version for production [35].
5. **Residency and lock-in.** Jev is a closed, US-hosted API with undisclosed architecture and possibly subsidised prices [33][4]. Kev and Laya run locally, for example via Ollaya [33][50][14][60]. Treat price stability as a risk in business cases.
6. **Compliance.** GDPR Article 22 restricts decisions based solely on automated processing that have legal or similarly significant

effects on people [29]. The EU AI Act's obligations for stand-alone high-risk systems were postponed to 2 December 2027, and for AI embedded in regulated products to 2 August 2028 [27]. The postponement was made by a regulation that entered into force on 27 July 2026 [28]. The deadlines moved, but the obligations remain, so use cases such as HR screening or credit decisions need a human in the loop from the start [27][28].

Deeper: `research.md`, sections 14 and 15.

---

9. My take

Slide 18, reading time about 1 min

Jev packages an old idea: reduce the AI task to one typed decision and leave the rest to the big model. The ecosystem of the first two weeks shows that the idea is portable [52].

A confidence value without published calibration looks precise, and it is tempting to accept a decision without checking it. The community list already contains bots that let Jev decide stock and crypto trades [55]. A reasoning model lets me read why it decided as it did; a probability comes without an explanation.

---

10. The live demo: Kev on a MacBook

Slide 19, reading time about 2 min

Everything in the demo runs on one laptop, with no cloud call for the decisions. Kev runs locally as a System-One-compatible server on port 8009. Because it speaks the same API as Jev, the same scripts work against Laya or Jev by changing the URL, the model name and the API key [58][50]. The demo takes about 12 minutes.

Part 1: terminal triage (about 3 minutes). The integration is one POST request that carries the ticket as state plus four questions: two choices, a yes-or-no and a score. Four typed answers with probabilities come back in one call, with zero output tokens [58]. A second ticket is written in capitals and says URGENT although the problem is cosmetic. The urgency the model assigns to this ticket shows whether it follows the content. The application defines the allowed answers and the model only picks [10].

Part 2: SAP CAP incidents (about 5 minutes). The urgency question moves into SAP's reference app for incident management [58] [59]. In the original a handler sets urgency to high when the title matches the regex "urgent" [59]. The demo changes it in three places:

- The options come from the CDS enum, so the model can only return values that exist in the data model.
- A threshold decides whether the code acts. Below it the app keeps the default and logs a manual review.
- The old regex stays as fallback, so the app's tests stay green [58].

Three incidents show the difference. "Not urgent: please send a copy of my invoice" makes the regex fire because the word is in there. A report about smoke that does not use the word slips past it. The log shows regex and Kev side by side [58]. The threshold of 0.5 is set by hand. In a project it would be calibrated on client data in shadow mode. Kev's confidence is normalised, so with three options a threshold of 0.5 corresponds to a raw probability of about 0.67 [58].

Part 3: agent router (about 4 minutes). Ten coding tasks for exactly this app. Kev answers two questions per task: which tier, mapped to Haiku, Sonnet or Opus, and whether the task is risky, meaning authentication, personal data or deletion. Risky tasks go to the largest model plus a human. If Kev is unsure, the task moves up one tier [58]. The savings depend on the task mix, and whether quality holds needs an evaluation, which matches the cascade study that could not yet prove the benefit [23]. OpenRouter now sells the same idea as a product, the Jev Router, which picks model and reasoning effort per request [61].

If Kev is down, part 2 still runs on the regex fallback and the log says that Kev is unreachable [58]. Kev also has a browser playground as a second fallback [56].

---

11. Questions to take away

Slide 20, reading time about 1 min

Two questions for the discussion, and for your own projects:

1. Where in our current projects are we already making System-1 decisions with expensive LLMs, and would a typed decision layer help?
2. If a model decides with 0.97 confidence and is wrong, who owns that decision: we, the client or the vendor?

12. Background chapter

Backup slides 22 to 24, reading time about 6 min

This chapter covers the questions that come up most often after the talk. It is optional reading.

RLHF and InstructGPT: where Almeida comes from

Reinforcement learning from human feedback (RLHF) goes back to a 2017 paper by Christiano and colleagues from OpenAI and DeepMind. The starting point was a practical problem: for many tasks it is hard to write down a reward function that says what good behaviour is. So non-expert humans compared pairs of short clips of an agent's behaviour and said which one was better. One model learned the goal from these preferences, and a separate reinforcement-learning step learned the behaviour to reach it. Feedback on about 0.1 percent of the agent's interactions was enough to solve Atari games and simulated robot tasks [37].

InstructGPT applied this idea to GPT-3. The paper was submitted in March 2022, and Diogo Almeida is its fourth author [38]. Its starting observation is that making a language model bigger does not make it better at following what a user wants; large models can be untruthful, toxic or unhelpful [38]. The method has three stages:

1. **Human demonstrations.** Labelers write examples of the desired behaviour, and GPT-3 is fine-tuned on them with supervised learning [38].
2. **Human rankings.** Labelers rank several model outputs for the same prompt. The rankings train a separate reward model that predicts which answer humans prefer [38][40].
3. **PPO.** The language model is then optimised against that reward model with the reinforcement-learning algorithm Proximal Policy Optimization [40].

In human evaluations, outputs of the 1.3-billion-parameter InstructGPT were preferred over those of the 175-billion-parameter GPT-3, although it is 100 times smaller [38]. ChatGPT followed in November 2022 and, according to OpenAI's announcement, was trained with the same methods and slightly different data collection [39].

Why this matters for Jev. RLHF optimises a model for what human raters like to read. Almeida's critique targets exactly that. In Forbes he said the industry has become superhuman at pleasing humans [7]. Jev's training method RLCD echoes the name of RLHF: reinforcement learning is still the tool. The signal is described as calibrated decisions, and the training data is fully synthetic [2]. How exactly this works is unpublished, so the comparison is our interpretation. A one-sentence version for clients: chat models are trained on human ratings of their text, decision models on calibrated decisions [2]. Whether their probabilities deserve trust has to be tested on the client's own data.

System 1 and System 2: what the metaphor says and where it stops

Daniel Kahneman, psychologist and Nobel laureate in economics, popularised the distinction in his 2011 book *Thinking, Fast and Slow* [42][46]. The labels come from the psychologists Keith Stanovich and Richard West [44].

	System 1	System 2
Character	automatic, fast, little effort, no sense of voluntary control [43]	effortful, slow, attention-hungry [43]
Examples	answering 2 + 2, turning towards a sudden sound, reading words [44]	counting how often the letter "a" appears on a page, checking a logical argument [43]
Role	produces impressions and feelings	builds thoughts step by step and can override System 1 [43]

Where it fits Jev. Jev plays System 1: it returns a fast, cheap impression without a visible chain of reasoning [5]. The surrounding code, a reasoning model or a human plays System 2, and the confidence threshold is the trigger that calls it in [11]. Kahneman lists counting letters on a page as a System-2 task, and TypeSafe advises keeping counting and arithmetic out of Jev [43][36][3].

When can intuition be trusted? A 2009 paper by Kahneman and Gary Klein asks when intuition can be trusted. They concluded that intuition is only trustworthy in predictable environments where the person had the chance to learn the regularities, and that subjective confidence is not a reliable sign of accuracy [45]. This matches the Jev evidence: on unfamiliar support tickets with an unseen priority rule it was right 44.7 percent of the time while claiming 74 percent [19]. The consequence is the same as for people. Trust the fast judgement only where the task is regular and where you have feedback to check it against, and send everything else to slow, deliberate processing. Kahneman and Klein's questions are a lightweight checklist: is the environment regular, is there feedback, and how costly is overconfidence? Using them this way is our own synthesis and has not been validated.

Caveat. The two-system model is contested in psychology. Melnikoff and Bargh argued in 2018 that dividing mental processes into two types lacks empirical support [47]. Pennycook and colleagues replied that the critique targets a view that current dual-process theorists have already given up [48]. System 1 works as a metaphor for the product category. It says nothing about how Jev works inside, and its architecture is undisclosed [33].

DistilBERT: the small encoder behind the pattern

DistilBERT was published in 2019 by Hugging Face researchers as a smaller version of Google's BERT [41]. It is an encoder: it reads the whole input at once and turns it into a representation instead of generating text token by token. It was created by knowledge distillation. A large teacher model (BERT) supervises a smaller student during pre-training, so that the student learns to imitate it [41].

	DistilBERT compared with BERT
Parameters	66 million against 110 million
Size	40 % smaller
Language understanding	keeps 97 %
Speed	60 % faster

All figures are from the authors [41].

A fine-tuned encoder like this works as a typed decision model. A small output layer maps the representation to probabilities over a fixed set of labels, so it is fast, cheap and cannot answer outside the label set. That is the same list of strengths as Jev's. The weakness is the flip side: it needs labelled data for each task and must be retrained when the categories change, while Jev works zero-shot on any schema you define at call time [21]. In the same study the encoder won with labels and Jev won without them [21]. DistilBERT was also the scorer inside FrugalGPT [25].

The choice for a client depends on their situation. With stable categories, thousands of labelled examples and strict residency requirements, a small encoder on their own infrastructure fits: in the Banking77 study it scored 0.933 at 9 milliseconds and no per-call cost, against 0.832 for Jev [21]. With changing categories or no labels, a zero-shot decision model can start without labelled data [21]. A combination is plausible and is our own suggestion, not an established result: start zero-shot, collect the decisions and human corrections as labels, and later replace the model by a distilled small one for the high-volume cases. This corresponds to the "LLM approximation" idea in the FrugalGPT paper [24].

Deeper: `research.md`, sections 5, 6 and 10.

13. Status and what may still change

Several facts in this article moved during the two weeks it covers, and some will move again before or after the talk:

- **Access.** Open to all around 21 September, invite-only again on 27 September [64][69].
- **Versions.** Jev is at 1.13, and TypeSafe publishes weaknesses per version [61][3].
- **Ecosystem.** New Jev-compatible models and runtimes appeared almost daily. Treat every self-reported benchmark as a claim until someone else has re-run it [52].
- **Benchmarks.** The independent numbers come from small studies. The same test measured Jev at 0.832, 0.87 and 0.78 on different Banking77 setups [21][62][22].

`research.md` keeps a dated news log for updates, and its section on contradictions lists where sources disagree.

Where to read more

Topic	`research.md`
Interface, prices, primitives	Section 3
Company, founder, training, architecture	Section 4
InstructGPT and RLHF	Section 5
Kahneman's two systems	Section 6
Launch dynamics and ecosystem	Sections 7 and 9
What "cannot hallucinate" means	Section 8
DistilBERT and small encoders	Section 10
Independent evaluations	Section 11
Calibration	Section 12
Criticism and counter-positions	Section 13
Consulting relevance, regulation	Section 14
Contradictions between sources	Section 15
Open questions	Section 16
Dated updates	Section 17

Sources

Numbering as in `research.md`. Only sources cited in this article are listed; the research document has the full list. All accessed between 24 and 27 September 2026.

[2] T. Fernholz, "A new kind of AI model from a ChatGPT inventor is thrilling developers", TechCrunch, 18 Sep 2026. https://techcrunch.com/2026/09/18/a-new-kind-of-ai-model-from-a-chatgpt-inventor-is-thrilling-developers/

[3] TypeSafe AI, "Jev 1.13 jaggedness" (documentation), last reviewed 17 Sep 2026. https://docs.typesafe.ai/model-jaggedness/jev-1.13

[4] MarkTechPost, "TypeSafe AI Releases Jev: A System One Model That Returns Typed, Calibrated Decisions Instead of Text", 19 Sep 2026. https://www.marktechpost.com/2026/09/19/typesafe-ai-releases-jev/

[5] Firecrawl, "What Is Jev? Inside TypeSafe's Decision-Only AI Model and Its Developer Use Cases", Sep 2026. https://www.firecrawl.dev/blog/what-is-jev

[6] The Next Web, "The fastest-adopted model on Vercel this year cannot write a word", Sep 2026. https://thenextweb.com/news/typesafe-jev-decision-model-vercel-fastest-adopted

[7] Wikipedia, "Jev (AI model)", version of 24 Sep 2026. https://en.wikipedia.org/wiki/Jev_(AI_model)

[8] Tom's Hardware, "TypeSafe AI's Jev offers an alternative to LLMs that claims to be 193x faster and 445x cheaper", Sep 2026. https://www.tomshardware.com/tech-industry/artificial-intelligence/typesafe-ais-jev-offers-an-alternative-to-llms-that-claims-to-be-193x-faster-and-445x-cheaper-system-one-type-model-is-bespoke-for-probabilistic-decision-making

[9] You.com, "What Is Jev? TypeSafe AI's System One Model Explained", Sep 2026. https://you.com/resources/what-is-jev

[10] Vercel, "What is Jev, TypeSafe AI's System One model?", Sep 2026. https://vercel.com/i/what-is-jev

[11] N. Torres, "Jev (TypeSafe) explained: the AI model that returns decisions instead of text", Sep 2026. https://ntorres.dev/blog/jev-typesafe-system-one-model

[12] Nandakishor M., "SalesRLAgent: A Reinforcement Learning Approach for Real-Time Sales Conversion Prediction and Optimization", arXiv:2503.23303, 30 Mar 2025. https://arxiv.org/abs/2503.23303

[13] DeepMost Innovations, "sales-conversion-model-reinf-learning" (model card), Hugging Face, 2025. https://huggingface.co/DeepMostInnovations/sales-conversion-model-reinf-learning

[14] TechAIWire, "Laya is a 421M open-weights answer to Jev", DEV Community, Sep 2026. https://dev.to/techaiwire/laya-is-a-421m-open-weights-answer-to-jev-4n97

[15] eesel AI, "Laya AI: the open 33ms decision model that can't hallucinate", Sep 2026. https://www.eesel.ai/blog/laya-ai

[17] anisselbd, "jev-phishing-bench" (GitHub repository), Sep 2026. https://github.com/anisselbd/jev-phishing-bench

[18] XenoSpectrum, "Jev, the AI That Never Writes a Sentence: Cheaper and Faster Than Claude, But Accuracy Swings With How You Ask", Sep 2026. https://xenospectrum.com/en/jev-typesafe-bert-classifier-decomposition/

[19] The D*AI*LY Brief, "TypeSafe's Jev Scores 62.6% Asked Once and 95% Split Five Ways", Sep 2026. https://www.beri.net/article/typesafe-jev-typed-decision-model-calibration-decomposition-shadow-eval

[20] GDE, "Jev After Eight Days of Independent Tests: Level With Mid-Price LLMs, Behind the Frontier", DEV Community, 23/24 Sep 2026. https://dev.to/gde/jev-after-eight-days-of-independent-tests-level-with-mid-price-llms-behind-the-frontier-1kln

[21] ickma2311, "jev-baselines-eval: Pre-registered independent eval of TypeSafe Jev" (GitHub repository), 18 Sep 2026. https://github.com/ickma2311/jev-baselines-eval

[22] We Call Shotgun, "What Is TypeSafe Jev? Jev vs LLM Explained (2026 Guide)", Sep 2026. https://wecallshotgun.com/blog/what-is-typesafe-jev-vs-llm

[23] JevBench, "Methodology — Independent evidence for Jev", Sep 2026. https://jevbench.xyz/methodology

[24] L. Chen, M. Zaharia, J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance", arXiv:2305.05176, 9 May 2023. https://arxiv.org/abs/2305.05176

[25] Towards Data Science, "FrugalGPT and Reducing LLM Operating Costs", 2024. https://towardsdatascience.com/frugalgpt-and-reducing-llm-operating-costs-ff1a6428bf96/

[26] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On Calibration of Modern Neural Networks", Proceedings of ICML 2017, PMLR 70:1321–1330. https://proceedings.mlr.press/v70/guo17a.html

[27] Council of the EU, "Artificial intelligence: Council gives final green light to simplify and streamline rules" (press release), 29 Jun 2026. https://www.consilium.europa.eu/en/press/press-releases/2026/06/29/artificial-intelligence-council-gives-final-green-light-to-simplify-and-streamline-rules/

[28] Cloud Security Alliance Labs, "EU AI Act's High-Risk Deadline: Deferred, Not Cancelled" (research note), 1 Aug 2026. https://labs.cloudsecurityalliance.org/research/csa-research-note-eu-ai-act-high-risk-deadline-omnibus-20260/

[29] GDPRLocal, "Automated Decision Making: Overview of GDPR Article 22", updated Jun 2026. https://gdprlocal.com/automated-decision-making-gdpr/

[30] T. Balbi, "Jev AI Model: What It's For and When Not to Use It", Sep 2026. https://tiarebalbi.com/en/blog/jev-ai-model-cant-hallucinate-still-wrong

[31] "jevassert" (Python package, independent community tool), PyPI, Sep 2026. https://pypi.org/project/jevassert/

[32] Beam AI, "Jev by TypeSafe: A Decision Model for AI Agents", Sep 2026. https://beam.ai/agentic-insights/jev-typesafe-ai-agents

[33] Trending Topics, "Jev: A New AI Model That Refuses to Write Text Has Developers Hooked", Sep 2026. https://www.trendingtopics.eu/jev-a-new-ai-model-that-refuses-to-write-text-has-developers-hooked/

[34] zephel01, "I read 7 clones of Jev. Only the computational form was reproduced, and that computational form has existed since 2020", note.com, Sep 2026. https://note.com/zephel01/n/ne9a2c037e513?hl=en

[35] MarkTechPost, "A Coding Guide to TypeSafe AI Jev: Typed Decisions, Calibrated Confidence, and Speculative Fan-Out", 23 Sep 2026. https://www.marktechpost.com/2026/09/23/a-coding-guide-to-typesafe-ai-jev/

[36] F. Copes, "A deep dive into Jev, TypeSafe's System One model", Sep 2026. https://flaviocopes.com/jev/

[37] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, D. Amodei, "Deep Reinforcement Learning from Human Preferences", Advances in Neural Information Processing Systems 30 (NIPS 2017); arXiv:1706.03741, 12 Jun 2017. https://papers.nips.cc/paper/7017-deep-reinforcement-learning

[38] L. Ouyang, J. Wu, X. Jiang, D. Almeida et al., "Training language models to follow instructions with human feedback", arXiv:2203.02155, submitted 4 Mar 2022 (NeurIPS 2022). https://arxiv.org/abs/2203.02155v1

[39] "Reinforcement Learning from Human Feedback" (book manuscript), arXiv:2504.12501, 2025; section "2023 to the Present: The ChatGPT Era", quoting OpenAI's ChatGPT announcement. https://arxiv.org/pdf/2504.12501

[40] AI Papers (aktagon), "Training Language Models to Follow Instructions with Human Feedback" (paper explainer), n.d. https://ai-papers.aktagon.com/papers/instructgpt/

[41] V. Sanh, L. Debut, J. Chaumond, T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter", arXiv:1910.01108, submitted 2 Oct 2019, revised 1 Mar 2020 (EMC² Workshop, NeurIPS 2019). https://arxiv.org/pdf/1910.01108

[42] D. Kahneman, *Thinking, Fast and Slow*, New York: Farrar, Straus and Giroux, 2011. ISBN 9780374275631. Book, no URL; content verified via [43] and [44].

[43] G. Mann, "Thinking, Fast and Slow by Daniel Kahneman" (book notes), n.d. https://grahammann.net/book-notes/thinking-fast-and-slow-daniel-kahneman

[44] LitCharts, "Thinking, Fast and Slow: Part 1, Chapter 1 summary", n.d. https://www.litcharts.com/lit/thinking-fast-and-slow/part-1-chapter-1

[45] D. Kahneman, G. Klein, "Conditions for Intuitive Expertise: A Failure to Disagree", American Psychologist 64(6):515–526, Sep 2009. Copy hosted by Charles University: https://dl1.cuni.cz/mod/resource/view.php?id=153128

[46] McKinsey Quarterly, "Strategic decisions: When can you trust your gut?" (interview with D. Kahneman and G. Klein), n.d. https://www.mckinsey.com/capabilities/strategy-and-corporate-finance/our-insights/strategic-decisions-when-can-you-trust-your-gut

[47] D. E. Melnikoff, J. A. Bargh, "The Mythical Number Two", Trends in Cognitive Sciences 22(4):280–293, 2018. https://acmelab.yale.edu/sites/default/files/melnikoff_bargh_2018_mythical_number_2_0.pdf

[48] G. Pennycook et al., "The Insidious Number Two", Trends in Cognitive Sciences, 2018, doi:10.1016/j.tics.2018.04.008. https://cse.buffalo.edu/~rapaport/575/The-Insidious-Number-Two_2018_Trends-in-Cognitive-Sciences.pdf

[49] A. Hume, "Jev's Architecture Unmasked", archerhume.com, 17 Sep 2026. https://archerhume.com/posts/jevs-architecture-unmasked/

[50] J. Palmer, "kev: Jev-like family of decision models built on top of Qwen3.5/3.8" (GitHub repository and README), accessed 21 and 27 Sep 2026. https://github.com/jaredpalmer/kev

[51] P. Pillitteri, "Open Source Clones of TypeSafe's Jev Arrive in a Week, Free on an RTX 3090", Sep 2026. https://pasqualepillitteri.it/en/news/17263/typesafe-jev-open-source-clones-free-rtx-3090

[52] Made with Jev, "Is Jev open source? No, but 35 open alternatives are", Sep 2026. https://madewithjev.com/open-source-jev

[53] System One Models, "Jev alternatives: open-source reproductions, local models and classifiers", Sep 2026. https://systemonemodels.org/examples/alternatives/

[55] yibie, "awesome-jev" (GitHub curated list), Sep 2026. https://github.com/yibie/awesome-jev

[56] Hugging Face Space "jaredpalmer/kev" (Kev browser playground), Sep 2026. https://huggingface.co/spaces/jaredpalmer/kev

[57] ScriptByAI, "9 Best Open-Source Jev Alternatives to Run Locally (2026)", Sep 2026. https://www.scriptbyai.com/jev-open-source-alternatives/

[58] Demo README "Demo: System-1-Entscheidungen lokal mit Kev" (jev-demo repository), provided by the user, Sep 2026. No public URL.

[59] SAP, "cap-js/incidents-app" (CAP reference application from the SAP BTP Developer's Guide), GitHub. https://github.com/cap-js/incidents-app

[60] Ollaya, "Ollaya · Run decision models locally" (project website), accessed 27 Sep 2026. https://ollaya.dev/

[61] OpenRouter, "TypeSafe: Jev Router" (model page), accessed 27 Sep 2026. https://openrouter.ai/typesafe/jev-router

[62] AbdelStark, "jev-benchmarks: Probability-aware evaluation for typed decision models" (GitHub repository, pilot v1), Sep 2026. https://github.com/AbdelStark/jev-benchmarks

[63] Supersonic Labs, "Introducing Julia 1", Sep 2026. https://supersoniclabs.ia.br/julia-1/

[64] Ground News, "What is Jev's new AI model and why are developers interested in it?" (news aggregation), Sep 2026. https://ground.news/article/what-is-jevs-new-ai-model-and-why-are-developers-interested-in-it

[65] Pydantic, "TypeSafe (Jev)" (Pydantic AI documentation), Sep 2026. https://pydantic.dev/docs/ai/models/typesafe/

[66] LangChain, "What Is Jev? A Guide to TypeSafe AI's System One Model" (blog), Sep 2026. https://www.langchain.com/blog/building-a-harness-with-jev

[67] J. Palmer, "jaredpalmer/kev-0.6b" (Hugging Face model card), Sep 2026. https://huggingface.co/jaredpalmer/kev-0.6b

[68] J. Palmer, "jaredpalmer/kev-27b" (Hugging Face model card), Sep 2026. https://huggingface.co/jaredpalmer/kev-27b

[69] TypeSafe AI, homepage with waitlist notice "TypeSafe is invite-only again", screenshot provided by the user, 27 Sep 2026. https://typesafe.ai/